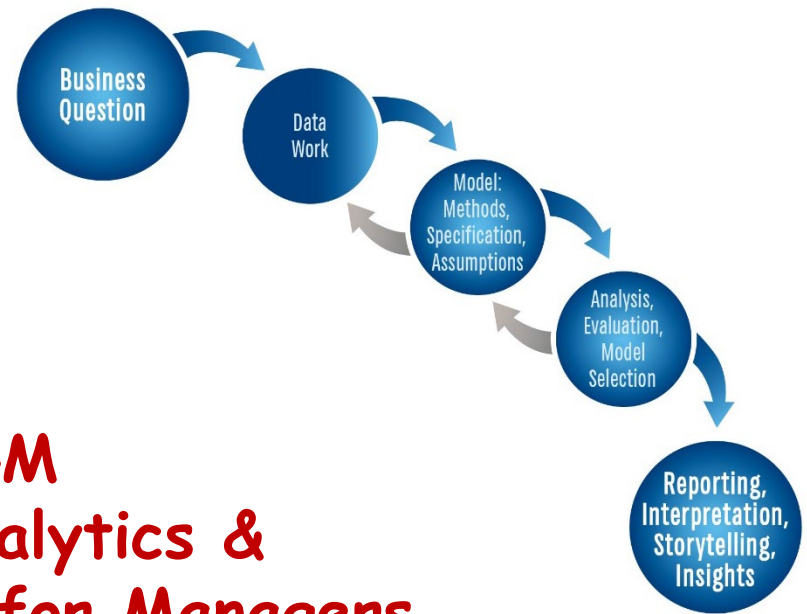




PAML4M
**Predictive Analytics &
Machine Learning for Managers**

8. Non-Linear Models

Use PAML4M_8R_NonLinearModels.Qmd

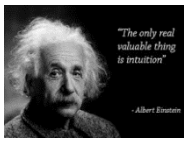
Prof. J. Alberto Espinosa

Last updated 8/1/2026



Non-Linearity Overview

$LI(x) - Y$ does not have a linear relationship with all X 's



Non-Linearity: Intuition

- **Linear model** → the **truth** is **never linear**, but the linearity assumption is often **good enough**: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \text{etc.}$
- In a linear model, the **effects** of the various variables are **additive**
- However, the relationship between **y** and the **x's** may not be linear. We discuss two types of non-linear relationships:
 - **Interactions**: The combined effect of **x₁** and **x₂** may be **multiplicative** rather than additive → the two predictors have an “**interaction**” (**x₁ * x₂**) effect → **enhancing** or **offsetting** each other's effect
 - **Polynomials**: The relationship between one predictor **x** and the response **y** may be **curvilinear** (e.g., quadratic, cubic, etc.)
 - **Piecewise or Splines**: The relationship between one predictor **x** and the response **y** may change **across segments** of the data

Non-Linear Model Types

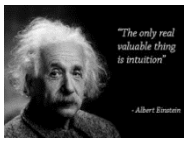
- There are many types of non-linear models
- We explore the following **non-linear models** in this section:
 - **B x C Interactions** → the effect of one predictor is affected by another; one predictor is **Binary** and the other is **Continuous**
 - **Polynomials** (the effect is quadratic, cubic, etc.)
 - **Piecewise** (the effect is different across sections of the data)
 - **Splines** (similar to piecewise)
- **Other** types we will **not** cover (see Goodies slides and R scripts):
 - **C x C Interactions** → the effect of one **Continuous** predictor is affected by another **Continuous** predictor (see Goodies)
 - **Smoothing Splines** (with “smoothing” parameter; see Goodies)

B x C Interaction Models

(Binary x Continuous)

(For Continuous x Continuous see Goodies)

LI(x) - Relationship between Y and X's are not linear



Interaction Models: Intuition

- Interaction models are very **popular** in predictive analytics
 - An interaction between any two variables x_1 and x_2 is modeled as a **multiplicative** effect: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2$
 - β_1, β_2 are the “**main**” effects; β_3 is the “**interaction**” effect
 - Interpreting interaction effects is **straightforward** if one of the predictors in the interaction term is **binary**, say x_2
 - But it is **not so easy** or intuitive if both variables are **continuous**
 - **Without** the **interaction** term, the equation above would be like having two **parallel** regression lines (i.e., dummy variable model), one line when $x_2 = 0$ and another when $x_2 = 1 \rightarrow$ the effects of x_1 and x_2 , which in this case would be “**additive**”
 - But **with** an **interaction** term, we also get two regression lines, but the lines are **no longer parallel**
- If **the interaction** effect has the **same sign** as the **main effect**, the interaction **enhances** the main effect; **otherwise**, it **offsets** it

A Simple Interaction Model

- Let's illustrate the concept of interaction effect with the earlier example for gas mileage of cars:

$$\text{mpg} = \beta_0 + \beta_{\text{Origin}}(\text{Origin}) + \beta_{\text{Year}}(\text{Year}) + \beta_{\text{Int}}(\text{Origin} \times \text{Year}) + \varepsilon$$

- As before, $\text{Origin} = 0$ for domestic cars and $\text{Origin} = 1$ for foreign cars

- β_{Year} and β_{Origin} are called “**main effects**” which are the same additive effects we saw earlier with dummy variable models.
- β_{Int} is the joint “**interaction effect**” of year and origin together

Important: when there are one or more interaction terms in a model, neither the main nor interaction effects can be interpreted individually. **They MUST be interpreted together.**

Interpretation of Main + Interaction Effects

Interaction Model Explained

$$\text{mpg} = \beta_0 + \beta_{\text{Origin}} (\text{Origin}) + \beta_{\text{Year}} (\text{Year}) + \beta_{\text{Int}} (\text{Origin} \times \text{Year}) + \varepsilon$$

- You can think of the model above as two separate regression models:

For **domestic** cars

$$\text{Origin} = 0 \rightarrow \text{mpg} = \beta_0 + \beta_{\text{Year}} (\text{Year}) + \varepsilon$$

For **foreign** cars:

$$\text{Origin} = 1 \rightarrow \text{mpg} = (\beta_0 + \beta_{\text{Origin}}) + (\beta_{\text{Year}} + \beta_{\text{Int}}) (\text{Year}) + \varepsilon$$

β_0 is the **intercept** for **domestic** cars;

$\beta_0 + \beta_{\text{Origin}}$ is the **intercept** for **foreign** cars

β_{Year} is the **main effect** of model **year** for **domestic** cars

$\beta_{\text{Year}} + \beta_{\text{Int}}$ is the **main effect** of model **year** for **foreign** cars

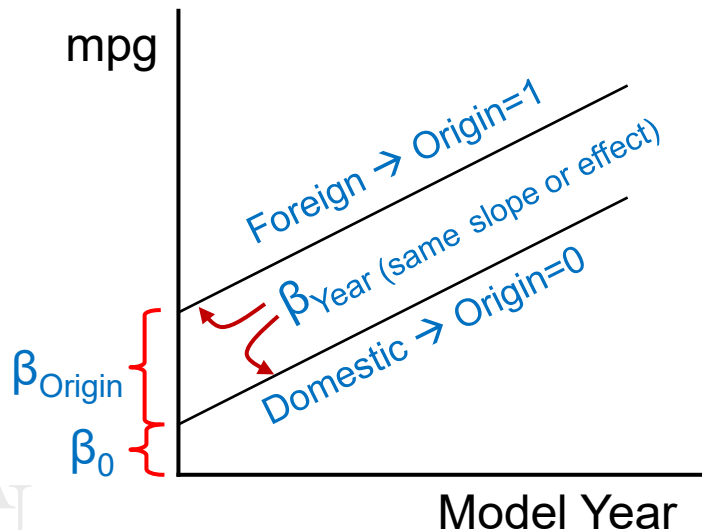
β_{Int} is the **difference** in effect of **year** (change in slope) between **domestic** and **foreign** cars $\rightarrow$ **interaction** effect

Interpretation

- Without an interaction term, the effects of Size and Origin are additive
- And the effect for Origin = 1 is a shift in the regression line with the same slope
- With an interaction term, the slope of the regression also changes
- β_{Year} → The effect of model year on mileage for domestic cars
- $\beta_{\text{Year}} + \beta_{\text{Int}}$ → The effect of model year on mileage for foreign cars
- β_{Int} → The effect of model year changes from domestic to foreign

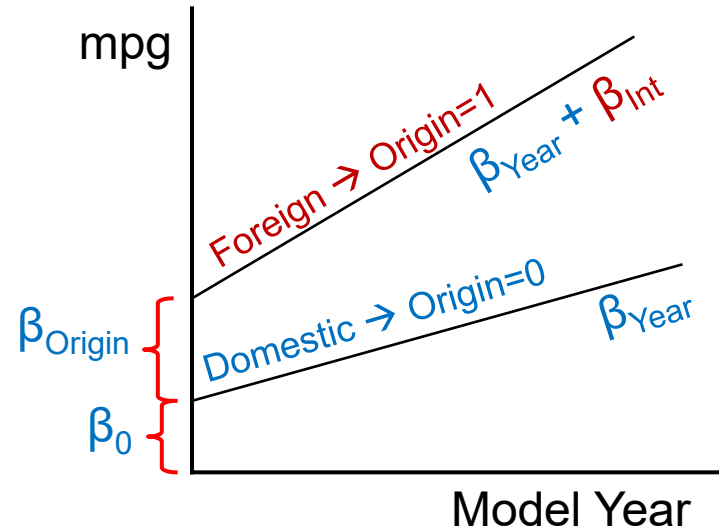
No Interaction Terms

$$\text{mpg} = \beta_0 + \beta_{\text{Origin}} (\text{Origin}) + \beta_{\text{Year}} (\text{Year}) + \varepsilon$$



With Interaction Term

$$\text{mpg} = \beta_0 + \beta_{\text{Origin}} (\text{Origin}) + \beta_{\text{Year}} (\text{Year}) + \beta_{\text{Int}} (\text{Origin} \times \text{Year}) + \varepsilon$$



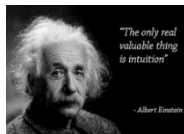
Binary x Continuous Interaction

- {library} Topic
Search keywords
- Binary x Continuous (B x C) Interaction Models
library(ISLR)

Polynomial Models

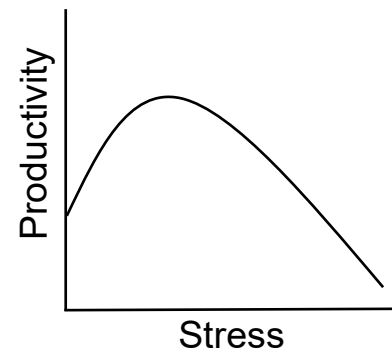
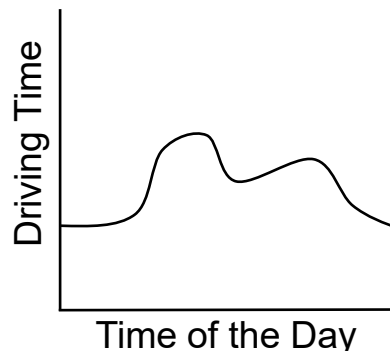
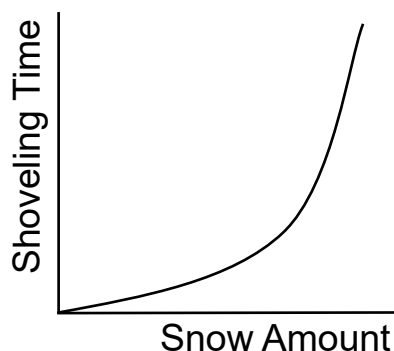
(Squared, Cubic, etc.)

LI(x) - Relationship between Y and X's are not linear



Polynomial Models: Intuition

- How long does it take to shovel 3", 6", 9", etc. of **snow**?
- How long does it take to **drive downtown** at a given time of the day
- Does **stress** lead to higher **productivity**?
- Sometimes relationships between variables are **not exactly linear**:

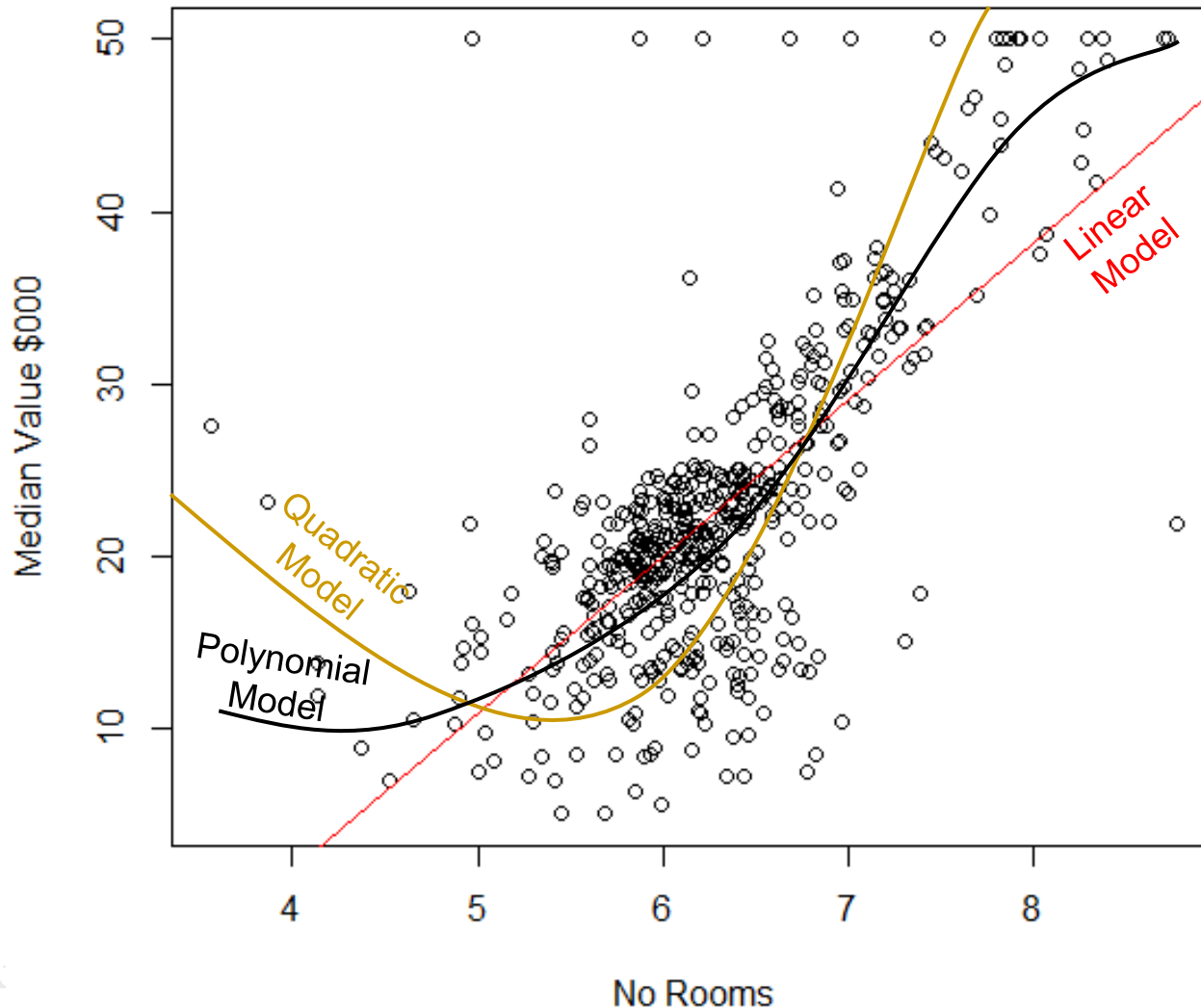


- In such cases, a **polynomial function** may be closer to the true relationship: $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_d x^d + \varepsilon$
- In general, **higher polynomials**:
 - Yield more peaks, valleys and inflections (i.e., **curvier functions**)
 - Produce more **overfitting** and other **dimensionality** issues
 - Yield **lower bias** and **higher variance** estimators
 - Are **harder to interpret**

Polynomial Models

(Boston {MASS} Data)

Regression for Boston House Values

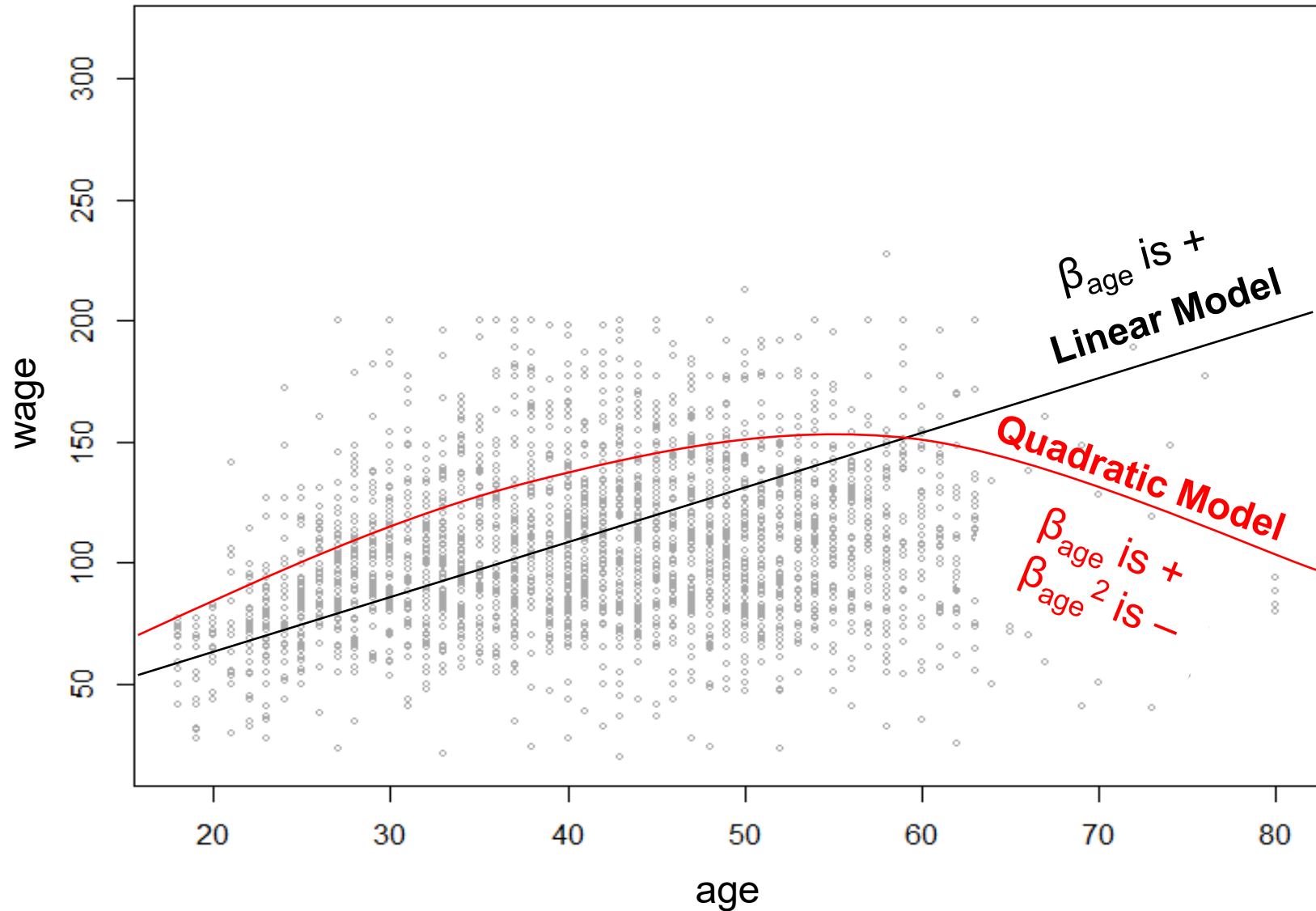


Questions

- **When** to use polynomial functions?
 - When there is a **business reason** to suspect a non-linear relationship
- What polynomial **degree** is optimal?
 - **Quadratic** is best for **interpretation**
 - **Higher** polynomials produce **better** training model **fit** and **less bias**
 - But generate **more variance** and **overfitting**
 - Especially at both ends of the curve → **“wagging the tail”**
 - Best polynomial should be selected using **cross-validation**

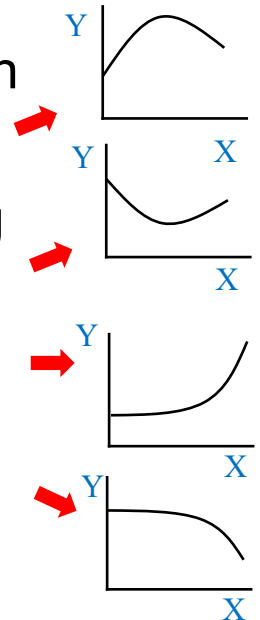
Polynomial Models

(Wage {ISLR} Data)



Polynomials: More Intuition

- Quadratic models are popular because they are **intuitive** while providing a **curvilinear fit**
- A quadratic model may be best if the effects appear to be:
 - ✓ **Positive** (negative) but diminishing up to a **peak** and then **downwards** → $\beta_x (+)$; $\beta_x^2 (-)$
 - ✓ **Negative** but diminishing, up to a **valley** and then turning **upwards** → $\beta_x (-)$; $\beta_x^2 (+)$
 - ✓ **Positive** and **augmenting** → $\beta_x (+)$; $\beta_x^2 (+)$
 - ✓ **Negative** and **diminishing** → $\beta_x (-)$; $\beta_x^2 (-)$
- A **higher polynomial** may be appropriate if the relationship between **y** and **x** is more **wavy**
- All candidate models should be evaluated with **ANOVA** or **cross-validation**



Fitting Polynomials

Fitting Polynomials

You can fit polynomials in 3 ways:

(1) $\text{lm}(y \sim x + I(x^2) + I(x^3) + \text{etc.}, \text{data} = \dots)$

$I()$ is the “as is” or “**identity**” function needed because $^$ has other interpretations in R

(2) $\text{lm}(y \sim \text{poly}(x, n), \text{data} = \dots)$

$\text{poly}()$ incorporates “**orthogonal**” polynomial terms up to the n^{th} degree in this case

(3) $\text{lm}(y \sim \text{poly}(x, n), \text{raw} = \text{T}, \text{data} = \dots)$

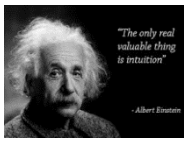
$\text{raw} = \text{T} \rightarrow$ same as (2), but it fits a regular polynomial, rather than orthogonal polynomials, thus yielding the same result as (1)



Example: Polynomial Model

- `{library}` Topic
Search keywords
- Polynomial Regression
`library(ISLR)`
Predictions and CV Testing with Polynomials

Piecewise Functions & Splines



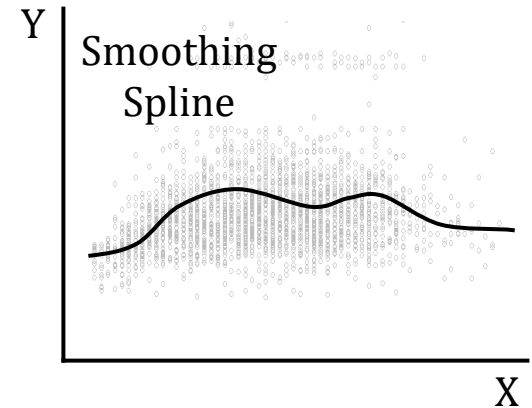
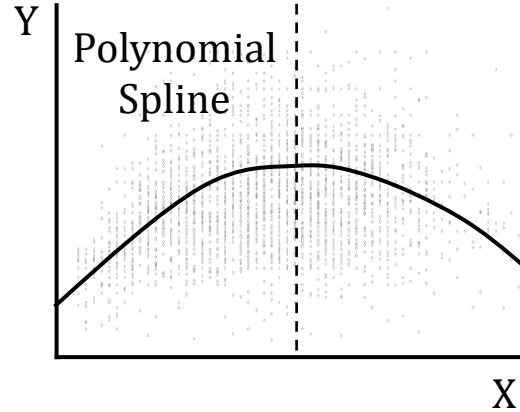
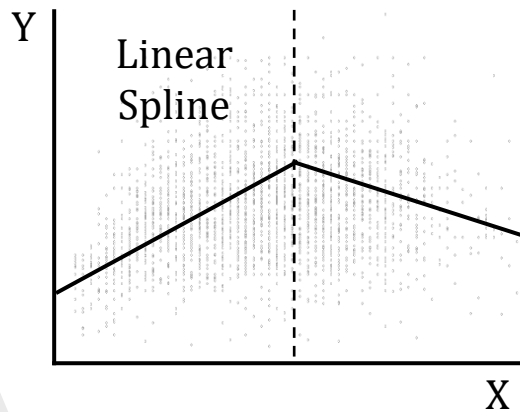
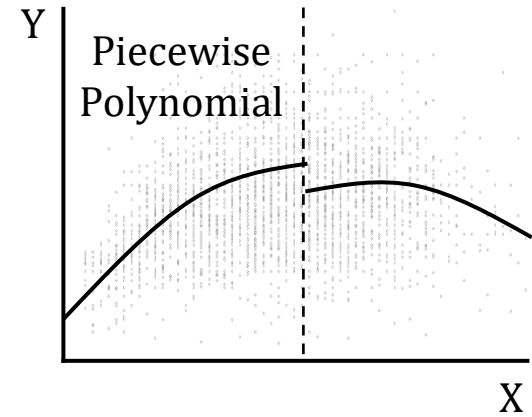
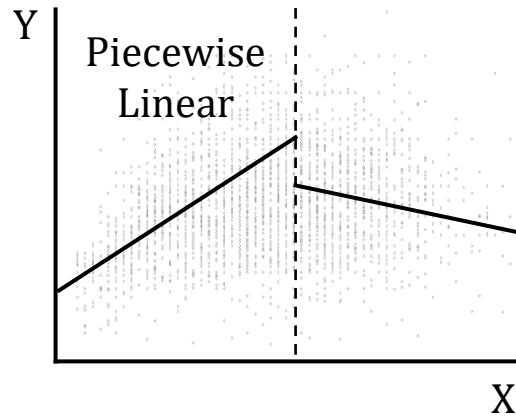
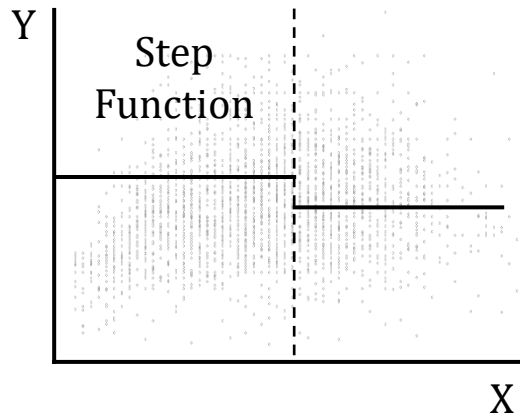
Piecewise Functions & Splines: Intuition

- **Polynomials** impose a mathematical **structure** on the model
- Complex mathematical structures generally provide **good fit** in the **training** data but often perform **poorly** with **test** data
- **Polynomials** often suffer from **over-identification** and high dimensionality
- They also tend to have high variance, especially at the tail ends of the curve (“**wagging the tail**”)
- Piecewise functions and splines offer a **simpler alternative** to complex polynomials → this simplicity often results in **lower variance**.
- All piecewise functions and splines:
 - Divide the data into various **segments**
 - **Fit** a different regression in **each segment**
 - A point where 2 segments connect is called a “**knot**”

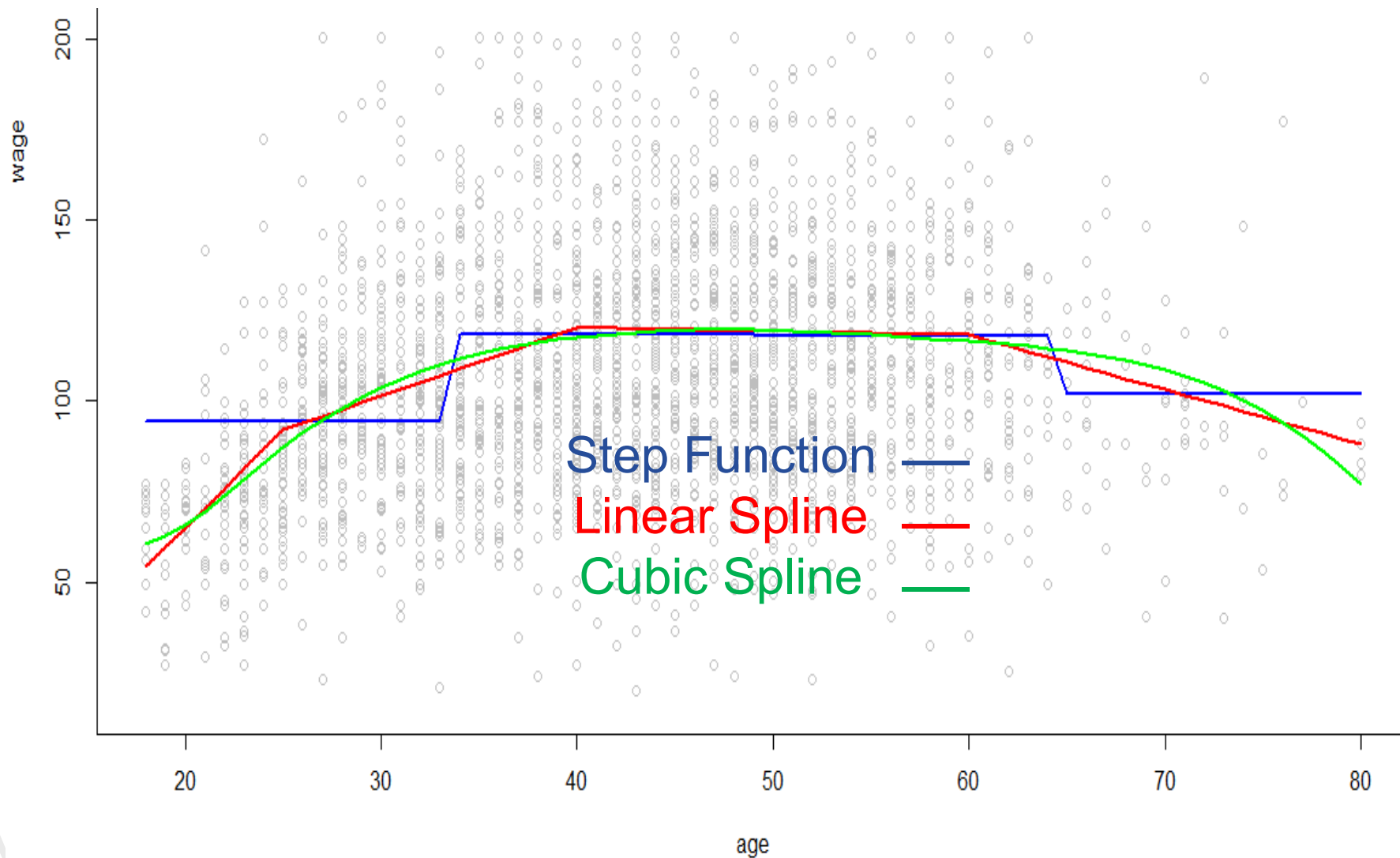
Piecewise Function & Spline Types

- The various types of piecewise functions depend on:
 - ✓ Which kind of **model** is **fit** in **each segment**, and
 - ✓ How the segments are **connected** at the **knots**
- **Step Function** – a **flat average** (no slope) in each section (not very useful, see R script)
- **Piecewise Linear** – a different **linear regression** in each section
- **Piecewise Polynomial** – a different polynomial in each section
- **Spline** – a piecewise function, but the segment regression lines are connected at each “**knot**” (i.e., **no jumps** at the knot)
- **Smoothing Spline** – a combination of **Ridge** (i.e., shrinkage) and **Spline**, which has the effect of **smoothing** the curve at the knot; how much smoothing is determined by a **tuning** parameter λ

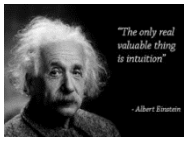
Piecewise & Spline Model Illustrations



Illustration



Piecewise Linear Models

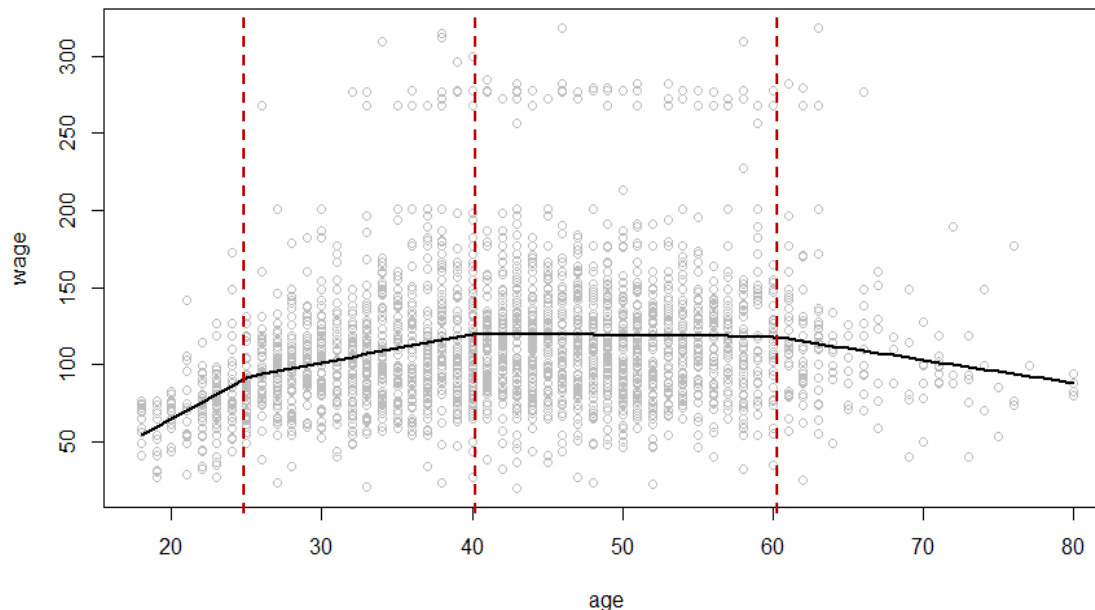


Piecewise Linear: Intuition

- If the data shows various upward or downward patterns at different data sections, **Piecewise Linear** may be the best alternative to **polynomials**:
 - The idea is to fit different **linear regression** models in **each section**
 - Piecewise linear models retain the **simplicity** of linear models and are **easier** to **interpret**
 - But because the slope of the regression can change at any selected point, it can provide **better** training **fit** with **less overfitting**.

Piecewise Linear Example $Wage\{ISLR\}$

- This method is very similar to a **linear spline**. We will see how to use spline functions for this later, but this manually coded piecewise linear method is more interpretable
- Say, you notice different roughly linear **wage patterns** in 4 segments changing at **ages 25, 40 and 60**
- The idea is to fit a **piecewise linear** function that give us **4** different regression **slopes** between these **3** particular “**knots**”





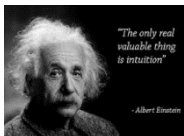
Specifying a Piecewise Linear Model

Specifying a Piecewise Linear Model

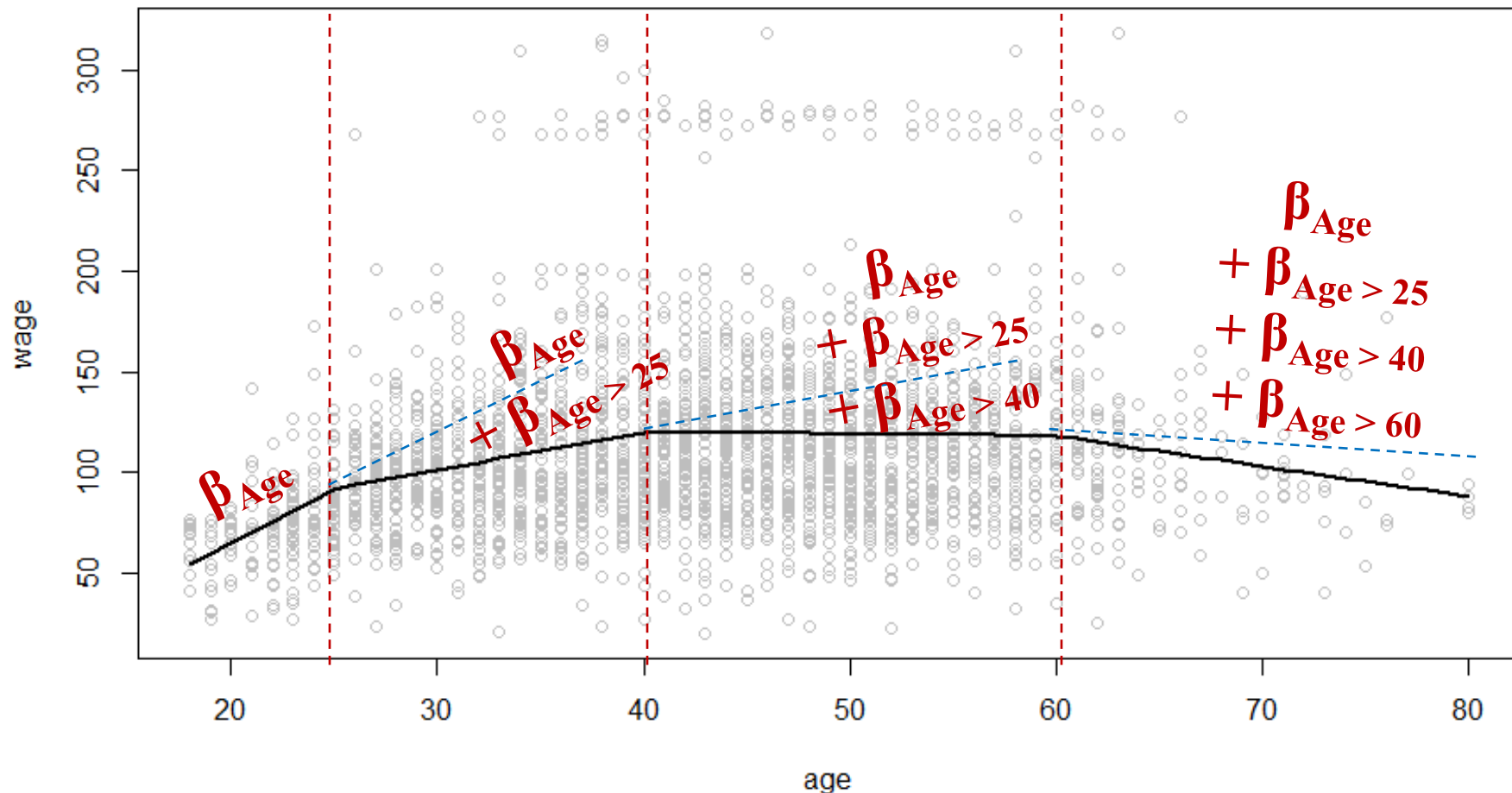
We do this by creating **binary & interaction** variables and specifying the model follows:

```
lm(wage ~ age + I( (age - 25) * (age > 25) ) +  
      I( (age - 40) * (age > 40) ) +  
      I( (age - 60) * (age > 60) ), data = Wage)
```

- The coefficient for `age` gives the linear effect of `age` on `wage`
- The function `I((age - 25) * (age > 25))` is an **interaction** variable:
 - ✓ It takes a value of **0** when `age < 25` (i.e., does **not change** the **slope** of the variable `age`,
 - ✓ It takes a value of **1** when `age >= 25` but the effect is not exactly of the variable `age`, but of ages over 25 (`age - 25`)
 - ✓ That is, as all interaction terms, its coefficient provides the **difference** in **slope** between age **beyond 25 years of age**
- Same thing for `(age > 40)` and `(age > 60)`



Graphical Illustration



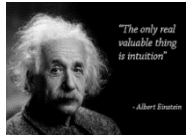
Note in this example (see R script) that β_{Age} is (+) but the remaining β 's are (-) $\rightarrow$ the effect of age goes down diminishes at ages 25, 40 and 60.



Example: Piecewise Linear

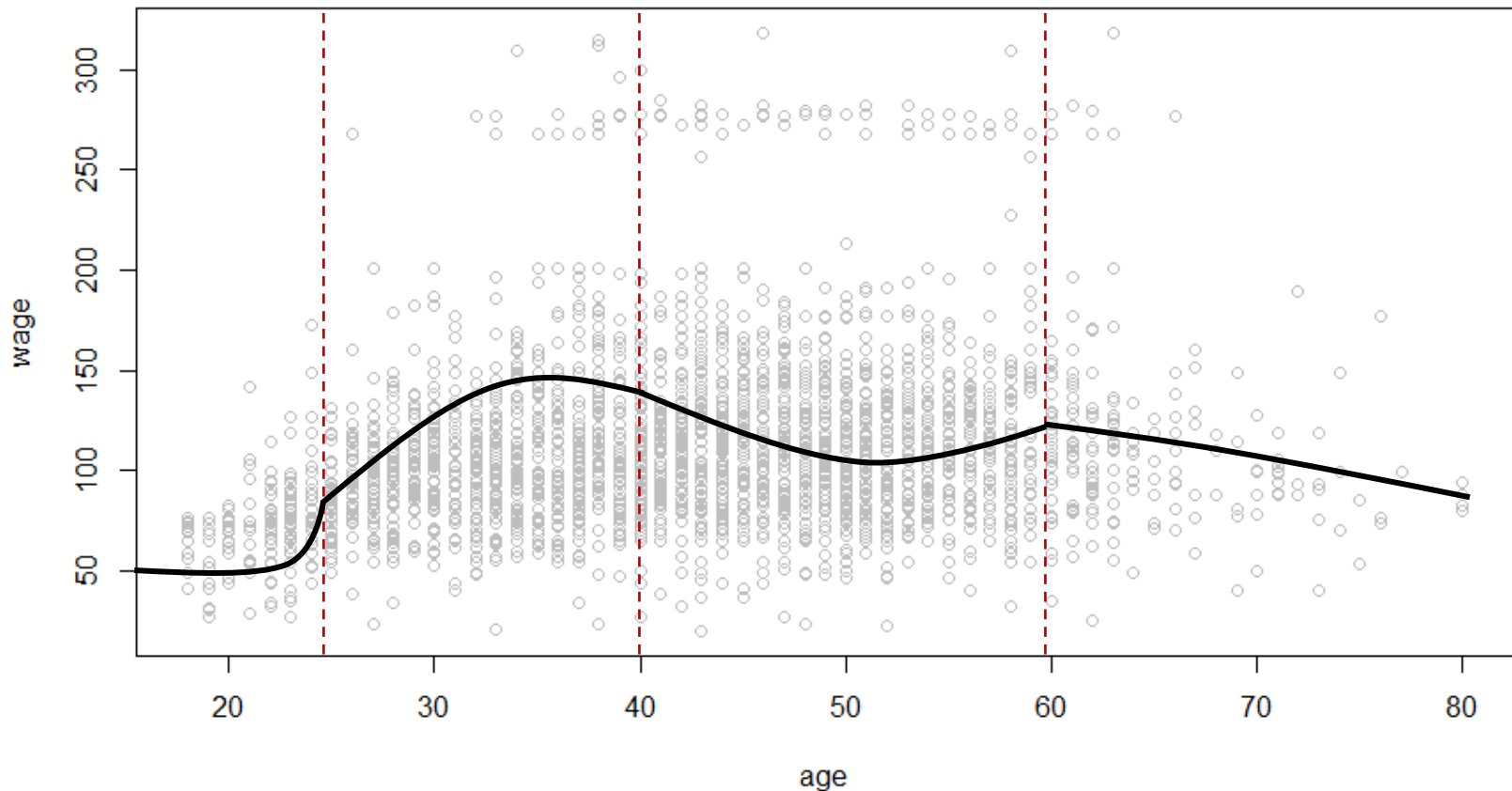
- `{library}` Topic
Search keywords
- Piecewise Linear Spline Regression
`library(ISLR)`
CV Error Testing

Spline (MARS) Models [FYI] (Multivariate Adaptive Regression Spline)



Spline Regressions: Intuition

A spline regression is very similar in concept to a piecewise regression, but you could fit in each segment, not just linear models, but polynomials too.



How Spline Regressions Work

- Not that great for interpretation, but are good for prediction
- The mathematics behind splines is explained in the textbook, they are complex, but it boils down to a simple concept:
 - For a given variable, start at the origin with a polynomial
 - Then add a “truncated” power function for values beyond the 1st knot:
 - ✓ 0 before the knot, i.e. for $x < c_1$ where c_1 is the 1st knot
 - ✓ A polynomial after the knot, i.e., $x \geq c_1$ but using $x - c_1$ instead of x which will cause the two polynomials to connect at the dot
 - ✓ In the prior example, the truncated function for the first knot at age = 25 in a cubic spline would be:
 - ❖ 0 for age < 25
 - ❖ $(\text{age} - 25)^3$ for age ≥ 25
 - Repeat this procedure at every knot
- This ensures that the various polynomial segments connect at the knot, thus yielding a continuous curve



Types of Spline Models

Splines come in different **flavors**:

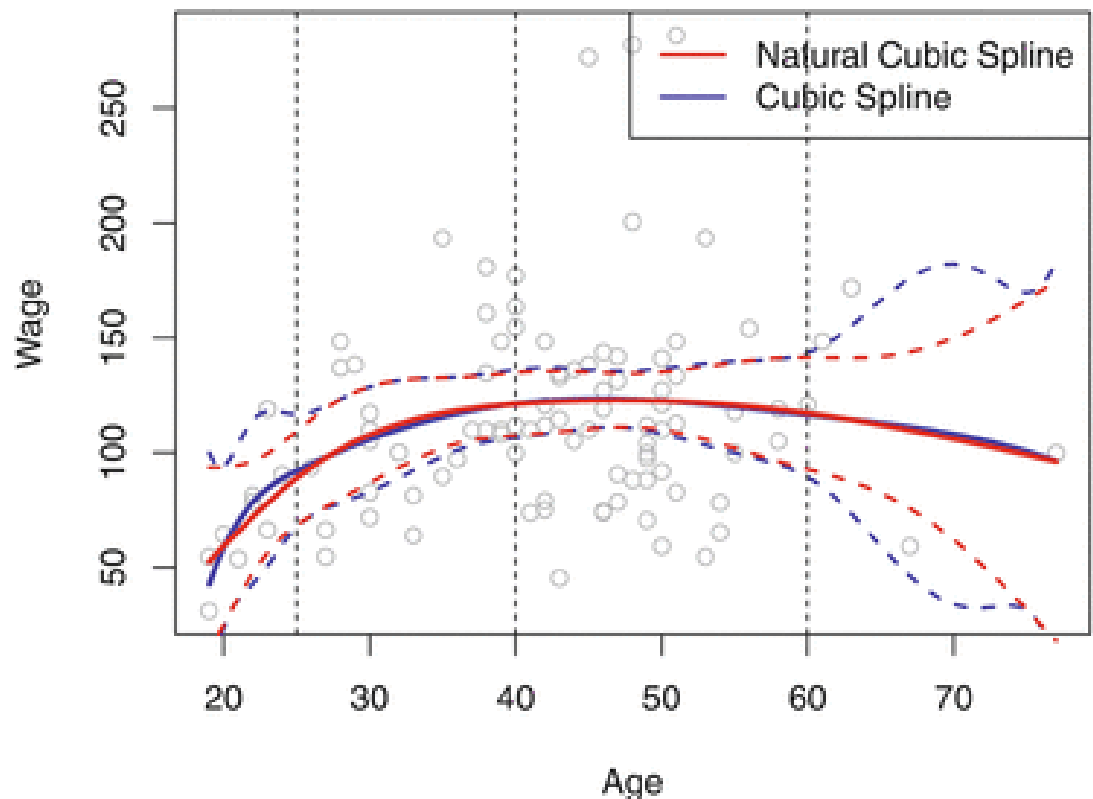
- ✓ **Linear Splines** – work just like the piecewise linear models, but using R `spline()` functions with `degree=1`
- ✓ **Polynomial Splines** – same as linear splines, but using the `degree=n` attribute to specify a polynomial degree
- ✓ **Cubic Splines** – a popular polynomial spline of `degree=3`, which is the **default** degree in most spline functions.
- ✓ **B-Splines** – or **Basis Splines**, are the same thing as polynomial splines, but there are some nuances associated with the name, and they differentiate from **N-Splines** discussed next.
- ✓ **N-Splines** – or **Natural Splines**, are similar to polynomial splines, except that the **first** and **last** segments are fitted with **linear** models. This often improves the predictive accuracy because the standard errors of the linear splines at the tail ends are typically lower than those of polynomial splines (less “**tail wagging**”).
- ✓ **Smoothing Splines** – splines with further smoothing at the knots. The degree of smoothing can be tuned (**See Goodies**).



Natural Splines [FYI]

More on Natural Splines

- Since we are using various polynomials to fit the splines, the splines will suffer from some of the typical **dimensionality** issues of polynomials, e.g., **high variance**, over-fitting
- Keeping the **dimensionality** of the segment splines **low** (i.e., cubic at the most) helps avoid some of these problems.
- However, splines are notorious for having **high variance** at the **outer ranges** of the data, where x is very **high** or very **low**.
- A **natural spline** helps correct for this problem to some extent by forcing the very **first** and **last segments** to a **linear** fit.



Tuning Spline Regressions

- As with other models, spline regressions need to be **tuned**
- The two most important **tuning parameters** are:
 1. The **number** of **segments** or **knots** to use
 - More segments/knots add **complexity** to the model and make it harder to interpret
 - More **knots** will yield a tighter **training fit**, but will not necessarily improve the **test MSE**
 - Each **knot** uses up **1 degree of freedom** in the model, so be mindful before adding more knots.
 - **Knots** can be **evenly spaced** or specifically **selected** based on business knowledge or observations of plots
 2. The **polynomial degree** or function to use in each segment
 - **Low degree** polynomials are preferred and more interpretable – i.e., **no higher than cubic** is recommended
- As with most other methods the best models and tuning parameters should be evaluated with **CV Testing**





Example: MARS Splines

- `{library} Topic`
Search keywords
- **Multivariate Adaptive Regression Spline (MARS)**
Linear B-Spline
Cubic B-Spline
Cubic N-Spline
Print ALL MSEs